VIP C2 PROGRAMME — FULL STACK DEVELOPMENT WITH MERN

Technology Stack

ShopEZ — MERN Stack E-commerce Application

Phase: Requirement Analysis Phase

Document Version: 1.1

Date: 11 June 2026

Prepared by: V S S S Manikanta

Classification: Project Documentation — For Academic Submission

Technology Stack

1. Introduction

This document provides a formal specification of the technology stack selected for the ShopEZ platform. Each technology selection is accompanied by its architectural role, justification, and a brief Architecture Decision Record (ADR) summarising the decision rationale and alternatives considered.

2. Integration Architecture Diagram

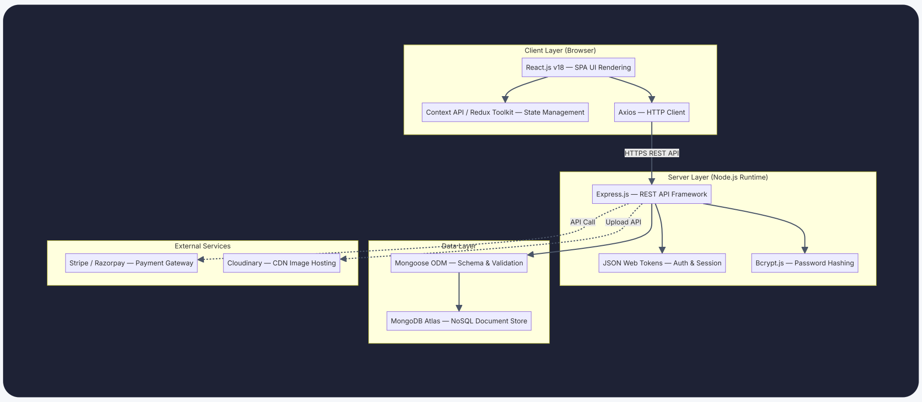

FIGURE 5.1 — TECHNOLOGY STACK INTEGRATION ARCHITECTURE DIAGRAM

3. Detailed Stack Breakdown

Layer	Technology	Version	Purpose & Justification
Frontend Framework	React.js	v18.x	Component-driven SPA architecture. Virtual DOM ensures efficient, targeted UI re-rendering. Hooks-based API enables clean functional component design.
State Management	Context API / Redux Toolkit	v2.x	Centralised global state for authentication context, shopping cart, and cached API data. Redux Toolkit reduces boilerplate via `createSlice` .
HTTP Client	Axios	v1.x	Promise-based HTTP client with request/response interceptors, automatic JSON serialisation, and configurable base URL. Preferred over native `fetch` for interceptor support.
Backend Runtime	Node.js	v18 LTS	JavaScript runtime enabling full-stack single-language development. Non-blocking, event-driven I/O suits concurrent API request handling.
Backend Framework	Express.js	v4.x	Minimal, unopinionated framework providing

Layer	Technology	Version	Purpose & Justification
			robust middleware composition, route parameterisation, and structured error handling for RESTful APIs.
Authentication	JSON Web Tokens (JWT)	RFC 7519	Stateless, self-contained bearer tokens signed with a server secret. Stored in HTTP-only cookies to mitigate XSS. Enables horizontal API scaling without shared session storage.
Password Security	Bcrypt.js	v5.x	Industry-standard adaptive hashing (salt rounds ≥ 10). Applied via Mongoose `pre-save` hook on the User model.
Database	MongoDB (Atlas)	v6.x	Document-oriented NoSQL store for flexible schema design. Hosted on Atlas for managed backups and auto-scaling.
ODM	Mongoose	v8.x	Schema-based ODM providing validation, type enforcement, virtual fields, populate references, and lifecycle hooks.

Layer	Technology	Version	Purpose & Justification
Payment Gateway	Stripe / Razorpay	Latest	PCI-DSS compliant payment APIs. No sensitive financial data stored on the ShopEZ server.
Media Hosting	Cloudinary	v2.x SDK	Provides automatic image compression, WebP conversion, and CDN delivery. Eliminates local filesystem dependency.
Deployment	Vercel (FE) / Render (BE)	—	Vercel provides React deployment with global CDN edge caching. Render hosts Node.js API with auto-deploy on Git push.

4. Architecture Decision Records (ADRs)

ADR-01: MongoDB over Relational Database

Field	Detail
Status	✅ Accepted
Context	The product catalogue requires a flexible schema — product attributes vary significantly across categories.
Decision	Adopt MongoDB as the primary data store.
Justification	Document-based storage natively accommodates variable product attributes without migration-heavy ALTER TABLE operations.
Consequences	No multi-document ACID transactions by default. Mitigated by Mongoose document-level atomicity for order creation and stock deduction.

ADR-02: JWT in HTTP-only Cookies over localStorage

Field	Detail
Status	✅ Accepted
Decision	Store JWT in HTTP-only, SameSite=Strict cookies rather than `localStorage` .
Justification	`localStorage` is fully accessible by JavaScript, making it vulnerable to XSS. HTTP-only cookies are inaccessible to client-side scripts.
Consequences	CSRF mitigation required via `SameSite=Strict` attribute and CORS configuration.

ADR-03: Cloudinary over Local File Storage

Field	Detail
Status	✅ Accepted
Decision	Use Cloudinary's cloud image management platform.
Justification	Provides automatic resizing, WebP conversion, and global CDN. Eliminates reliance on application server's ephemeral filesystem (Render).
Consequences	External API dependency and storage cost. Mitigated by enforcing file size limits in upload middleware.